

NEXTURN

Data Engineering Training

Scala and Apache Spark Bank Transaction Monitoring

Technical Documentation

Name: Chintha Vishnu Priya

Role: Data Engineering Trainee

Project: Bank Transaction Monitoring

Technologies: Scala 2.12.18, Apache Spark 3.5.6

Processing: Batch Processing, Stateful Streaming, Sliding Window

Monitoring: Account Frequency, Transaction Amount, Geographic Activity

Comapany: Nexturn India Private Limited

Mentor: Manisha Shah

TABLE OF CONTENTS

No.	Topic	Page
1	Introduction	6
2	Project Objectives	10
3	Project Scope	14
4	Technology Stack and Spark Dependencies	18
5	System Architecture and Processing Flow	22
6	Core Data Model	24
7	Batch Processing Module	27
8	Account Frequency Monitoring	32
9	High-Amount and Geographic Monitoring	35
10	Stateful Streaming Module	39
11	Sliding Window Processing	44
12	Final Monitoring Processor	48
13	Testing and Validation	52

TABLE OF CONTENTS

No.	Topic	Page
14	Pair RDD, Narrow and Wide Transformations	56
15	DAG, Stages and Shuffle	61
16	Broadcast Variables and Accumulator	65
17	Cache, Persist and Partition Management	68
18	DataFrame Analytics and Aggregations	71
19	RDD Lineage and Fault Tolerance	72
20	Final Results	75
21	Performance, Deployment and CI	77
22	Conclusion	79

LIST OF FIGURES

No.	Topic	Page
1	Bank Transaction Monitoring Dashboard Overview	7
2	Project Directory Structure	17
3	Project Build Environment and GitHub Actions Configuration	21
4	System Architecture of the Bank Transaction Monitoring System	23
5	BankTransaction Case Class Defining the Transaction Data Model	25
6	Sample Transaction Records from the Bank Transaction Monitoring Input Dataset	25
7	Batch Processing Module Execution Output	31
8	Account-Wise Transaction Frequency Analysis and High-Frequency Account Detection	34
9	Geographic Transaction Frequency Analysis and High-Frequency Location Detection	38
10	Stateful Streaming Execution Showing Cumulative Account Transaction State	42
11	Sliding Window Processing Using a 15-Second Window and 5-Second Slide	47
12	Final Monitoring Processor Execution Using Spark Submit	51
13	Successful GitHub Actions Build and Packaging Validation	55
14	Pair RDD Account-Frequency Aggregation and High-Frequency Account Detection	58
15	Spark DataFrame Schema and Account-Level Transaction Statistics	71
16	Bank Transaction Monitoring Dashboard Showing Final Pipeline Results	74
17	GitHub Actions CI Workflow Showing Successful Project Validation	78

LIST OF TABLES

No.	Table	Page
1	Technology Stack	18
2	Transaction Data Schema	24
3	Batch Execution Results	30
4	High-Amount Transaction Results	35
5	Geographic Monitoring Results	37
6	Combined Monitoring Results	38
7	Stateful Streaming vs Sliding Window	47
8	Monitoring Thresholds	49
9	Final Execution Summary	51
10	Input Validation Results	52
11	Narrow vs Wide Transformations in the Project	59
12	Broadcast and Accumulator Comparison	67

1.INTRODUCTION

The **Bank Transaction Monitoring** project is an end-to-end data engineering implementation developed using **Scala and Apache Spark**. The system processes structured banking transaction records and analyzes transaction activity to identify unusual patterns.

The monitoring system focuses on three primary dimensions: **account transaction frequency, transaction amount and geographic activity**. These dimensions provide different views of transaction behavior. Account frequency identifies accounts generating an unusually large number of transactions, amount monitoring identifies individual high-value transactions, and geographic monitoring identifies locations with unusually high transaction activity.

The project uses Apache Spark because transaction-processing workloads can contain large numbers of records and require distributed processing. Spark provides RDDs, Pair RDDs, DataFrames and Streaming APIs that allow the same transaction dataset to be processed using different approaches.

The implementation contains both **batch and streaming processing**. Batch processing analyzes the complete transaction dataset and produces deterministic monitoring results. Stateful streaming demonstrates how account-level information can be maintained across micro-batches using `updateStateByKey()`. A separate sliding-window processor demonstrates recent transaction activity using a **15-second window with a 5-second slide**.

The project also demonstrates important Spark concepts including Pair RDD operations, narrow and wide transformations, `reduceByKey()`, broadcast variables, accumulators, caching, partition management, DataFrame aggregation, DAG execution, shuffle, lineage and fault tolerance. These are not separate theoretical examples; they are demonstrated through the transaction-monitoring workflow.

The final monitoring pipeline was packaged and executed using `spark-submit`. The project also includes an HTML/CSS dashboard for presenting the monitoring results and a GitHub Actions workflow for automated compilation and packaging validation.

The validated dataset contains **20 valid transactions and 0 malformed transactions**. The monitoring process identifies **one high-frequency account, four high-amount transactions and one high-frequency location**

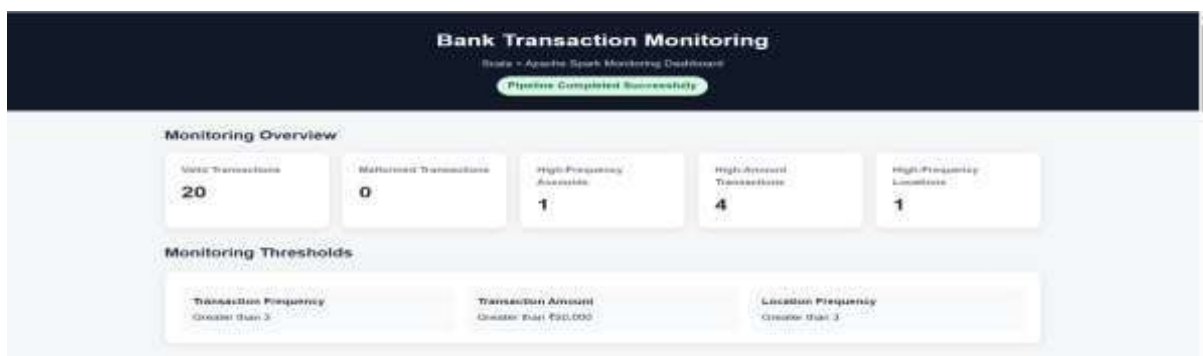


Figure 1. Bank Transaction Monitoring dashboard showing the final validated monitoring metrics.

1.1 PROJECT OVERVIEW

The project implements a Spark-based transaction monitoring pipeline in which transaction records move through several processing stages. The input CSV file is parsed and validated before the valid records are reused by different Spark processing operations.

The main processing path uses a cached transaction RDD. Pair RDD operations are then used to calculate transaction frequency by account and location. Individual transactions are filtered according to the high-amount threshold. The valid transaction data is also converted into a DataFrame to perform structured analytical operations.

The streaming component extends the project from static analysis to event-oriented processing. StreamingProcessor maintains account-level state across micro-batches, while WindowProcessor analyzes recent activity using a time-based window.

The final processor brings the main monitoring operations together and produces a summary that can be displayed through the project dashboard.

1.2 PROBLEM STATEMENT

Financial transaction systems can generate a large number of transactions across multiple accounts and locations. Manually examining each transaction is inefficient and makes it difficult to identify unusual activity quickly.

The objective of this project is therefore to implement a scalable transaction-monitoring workflow capable of analyzing transaction records and identifying:

1. Accounts with unusually high transaction frequency.
2. Individual transactions with unusually high amounts.
3. Locations with unusually high transaction activity.
4. Account-level activity across streaming micro-batches.
5. Recent transaction activity through sliding-window processing.

The project uses Spark transformations and aggregations to perform these operations while also demonstrating the execution concepts required for a practical Spark data-engineering workflow.

1.3 NEED FOR THE PROJECT

Transaction monitoring requires processing multiple dimensions of transaction activity. A single transaction may be normal when considered individually but may become significant when combined with other transactions belonging to the same account or location.

For example, an account with a large number of transactions within the monitored dataset may require additional attention even when the individual transaction amounts are relatively

small. Similarly, a location with an unusually high concentration of transactions may represent a pattern worth monitoring.

The project therefore uses multiple aggregation levels:

- **Transaction level** — high-amount monitoring.
- **Account level** — frequency monitoring.
- **Location level** — geographic monitoring.
- **Time level** — stateful and window-based streaming.

This multi-dimensional design is one of the main characteristics of the project.

2. PROJECT OBJECTIVES

The main objective of the **Bank Transaction Monitoring** project is to develop a practical data-processing pipeline using **Scala and Apache Spark** for identifying unusual transaction activity. The project combines batch processing, streaming processing and structured analytics within a single monitoring workflow.

2.1 Transaction Data Processing

The system reads banking transaction records from a CSV input source and converts each valid record into a structured BankTransaction object. The input is validated before the records are passed to the monitoring operations.

The processing pipeline is designed so that the same validated transaction data can be reused by multiple Spark operations.

2.2 Account Frequency Monitoring

The project monitors the number of transactions associated with each account.

A **Pair RDD** is created using the account ID as the key and 1 as the value for every transaction. The values are then aggregated using `reduceByKey()`.

The configured condition is:

Account transaction count > 3

Using the project dataset, **ACC001 has 6 transactions**, making it the identified high-frequency account.

2.3 High-Amount Transaction Detection

The system checks individual transaction amounts against a predefined monitoring threshold.

The configured condition is:

Transaction amount > ₹50,000

The implemented pipeline identifies four transactions above this threshold:

- TXN005
- TXN009
- TXN014
- TXN015

Therefore, the final monitoring summary contains **4 high-amount transactions**.

2.4 Geographic Activity Monitoring

The system also analyzes transaction activity according to location.

Transactions are grouped by their location and the number of transactions associated with each location is calculated. A location is considered high-frequency when its transaction count is greater than the configured threshold.

The configured condition is:

Location transaction count > 3

In the project dataset, **Hyderabad has 7 transactions** and is therefore identified as the high-frequency location.

2.5 Data Quality Monitoring

The project does not assume that every input record is valid. Transaction parsing includes validation so that malformed records can be identified separately from valid records.

An **accumulator** is used to maintain the malformed transaction count during processing.

For the validated dataset:

Valid transactions = 20

Malformed transactions = 0

2.6 Stateful Stream Processing

The project demonstrates stateful processing using **Spark Streaming** and `updateStateByKey()`.

The purpose is to maintain account-level transaction counts across incoming micro-batches. Instead of treating every micro-batch as an independent dataset, the processor maintains the previous account state and updates it when new transactions arrive.

The demonstrated streaming configuration uses a **5-second micro-batch interval**.

2.7 Sliding Window Processing

The project also demonstrates time-based monitoring through a sliding window.

The implemented window configuration is:

- **Window duration:** 15 seconds
- **Slide duration:** 5 seconds

This allows the processor to examine recent transaction activity repeatedly instead of considering only the complete historical input.

2.8 Structured Data Analytics

The project converts transaction records into a Spark DataFrame for structured analysis.

The DataFrame processing includes account-level statistics such as:

- Transaction count
- Total transaction amount
- Average transaction amount
- Maximum transaction amount
- Minimum transaction amount

Additional transaction-type and location statistics are also generated.

2.9 Spark Processing Concepts

The implementation demonstrates important Apache Spark concepts through the actual project:

- RDD processing
- Pair RDD
- map()
- filter()
- reduceByKey()
- Narrow transformations
- Wide transformations
- Shuffle
- repartition()
- coalesce()
- Broadcast variables
- Accumulators
- Cache and persistence
- DataFrames
- Spark Streaming
- Stateful processing
- Sliding windows
- DAG and stages

- Lineage and fault tolerance

The project brief specifically identifies these Spark concepts as part of the Bank Transaction Monitoring exercise.

2.10 Final Pipeline and Validation

The final objective is to integrate the monitoring operations into an executable Spark application.

The application is packaged using **sbt** and executed using `spark-submit`. The project also uses GitHub Actions to validate compilation and packaging and provides an HTML/CSS dashboard for presenting the final monitoring results.

The completed implementation therefore covers the complete flow:

**Transaction Input → Validation → Spark Processing → Monitoring Rules → Analytics
→ Streaming/Window Processing → Final Results → Dashboard**

3. PROJECT SCOPE

3.1 Scope of the Monitoring System

The Bank Transaction Monitoring project focuses on developing an end-to-end transaction monitoring system using **Scala and Apache Spark**. The system processes structured banking transaction records and identifies unusual activity based on predefined monitoring rules.

The monitoring system covers three major dimensions:

- **Account transaction frequency**
- **Transaction amount**
- **Geographic transaction activity**

The project combines batch processing, stateful streaming, sliding-window processing, and structured DataFrame analytics. The results are consolidated into meaningful monitoring outputs and presented through a dashboard for easier interpretation.

3.2 Batch Processing Scope

The batch processing module processes the complete transaction dataset available in CSV format. The module performs data ingestion and validation before applying the monitoring rules.

The batch processing scope includes:

1. Reading transaction records from the input CSV file.
2. Validating transaction fields and identifying malformed records.
3. Maintaining a count of malformed transactions using an **accumulator**.
4. Creating account-level transaction counts using **Pair RDD** operations.
5. Detecting accounts whose transaction frequency exceeds the defined threshold.
6. Identifying high-value transactions.
7. Grouping transactions by geographic location.
8. Detecting locations with unusually high transaction activity.
9. Converting transaction data into a Spark DataFrame.
10. Performing account-level and transaction-level aggregations.
11. Demonstrating partition management using `repartition()` and `coalesce()`.

The batch module therefore provides the main deterministic analysis of the available transaction dataset.

3.3 Streaming Processing Scope

The streaming component demonstrates how the monitoring system can process transactions arriving over multiple micro-batches.

A file-based Spark Streaming approach is used to simulate continuously arriving transaction data. The system maintains account-level transaction counts across micro-batches using **updateStateByKey()**.

The streaming scope includes:

- Processing incoming transaction files as micro-batches.
- Extracting account identifiers from incoming transactions.
- Maintaining cumulative account-level state.
- Updating transaction counts whenever new records arrive.
- Identifying accounts whose cumulative activity crosses the monitoring threshold.
- Using checkpointing to support recovery of streaming state.

This component demonstrates the difference between processing a fixed historical dataset and continuously maintaining information about incoming transactions.

3.4 Sliding Window Processing Scope

The sliding-window component focuses on **recent transaction activity** rather than cumulative activity.

The project uses a **15-second window with a 5-second slide interval**. This allows the system to repeatedly evaluate transactions occurring within the most recent time period.

The window-processing scope includes:

- Receiving transaction records through Spark Streaming.
- Maintaining a recent activity window.
- Removing transactions that fall outside the active window.
- Re-evaluating account activity at each slide interval.
- Detecting short-term bursts of transaction activity.

Sliding-window processing is useful for monitoring scenarios where recent activity is more important than the complete historical transaction count.

3.5 Dashboard Scope

A browser-based monitoring dashboard is included to provide a visual representation of the final monitoring results.

The dashboard presents key metrics such as:

- Total valid transactions
- Total malformed transactions
- High-frequency accounts
- High-amount transactions
- High-frequency geographic locations
- Monitoring threshold values
- Account-level transaction statistics
- Geographic monitoring results

The dashboard provides a simple operational view of the processing results and makes the output easier to understand than raw terminal logs.

3.6 Project Boundaries

The project is designed as a practical educational implementation of a bank transaction monitoring pipeline. The system operates on structured transaction data stored in CSV format.

The current implementation does **not** include:

- External banking systems
- Persistent production databases
- Real-time message brokers such as Kafka
- Actual banking authentication systems
- Production financial decision-making
- Complex machine-learning fraud detection

Instead, the project concentrates on demonstrating **Apache Spark data-processing concepts** through a realistic transaction-monitoring use case.

The implementation provides a foundation that can later be extended with additional data sources, message brokers, databases, advanced fraud-detection rules, and machine-learning models.

```

vishnupriya@vishnupriya:~/bank-transaction-monitoring$ cd ~/bank-transaction-monitoring

find . -maxdepth 4 \
  -not -path './.git*' \
  -not -path './target*' \
  -not -path './project/target*' \
  -not -path './data/checkpoint*' \
  -not -path './.bsp*' \
  -not -path './.idea*' \
  -not -path './.vscode*' \
  -type f | sort
./README.md
./build.sbt
./data/input/stream/batch1.csv
./data/input/stream/batch2.csv
./data/input/transactions.csv
./data/input/window/window1.csv
./data/input/window/window2.csv
./docs/dashboard-preview.png
./docs/index.html
./outputs/batch-output.txt
./outputs/spark-submit-output.txt
./outputs/streaming-output.txt
./outputs/window-output.txt
./project/build.properties
./src/main/scala/BatchProcessor.scala
./src/main/scala/FinalMonitoringProcessor.scala
./src/main/scala/Models.scala
./src/main/scala/StreamingProcessor.scala
./src/main/scala/WindowProcessor.scala
vishnupriya@vishnupriya:~/bank-transaction-monitoring$ |

```

Figure 2. Project directory structure of the Bank Transaction Monitoring system.

4. TECHNOLOGY STACK AND SPARK DEPENDENCIES

4.1 Technology Stack

The Bank Transaction Monitoring system is implemented using **Scala and Apache Spark**. The selected technologies support distributed data processing, streaming, structured analytics, application packaging, version control, and project validation.

The major technologies used in the project are listed below.

Technology	Version / Usage
Scala	2.12.18 — Application programming language
Apache Spark Core	3.5.6 — RDD and distributed processing
Spark SQL	3.5.6 — DataFrame and structured analytics
Spark Streaming	3.5.6 — Micro-batch and streaming processing
Java OpenJDK	17.0.20 — Runtime environment
sbt	2.0.7 — Build and package management
Linux / WSL2	Development and execution environment
CSV	Input transaction data format
HTML/CSS	Monitoring dashboard
Git + GitHub	Version control and project hosting
GitHub Actions	Continuous integration and validation

Table 1. Technology Stack

4.2 Scala

Scala 2.12.18 is used as the primary programming language for the project.

Scala provides concise syntax and integrates directly with Apache Spark. The project uses Scala to implement the transaction model, batch processor, streaming processor, window processor, and final monitoring pipeline.

The application is organized into separate Scala source files so that different processing responsibilities can be maintained independently.

4.3 Apache Spark

Apache Spark 3.5.6 is the central processing framework used in the project.

Spark provides the distributed processing capabilities required for transaction analysis. The project demonstrates several Spark components, including:

- RDD
- Pair RDD
- Spark transformations
- Spark actions
- Spark Streaming
- DataFrames
- Aggregations
- Partition management
- Broadcast variables
- Accumulators
- Caching

The project therefore uses Spark not only for basic data processing but also to demonstrate important distributed-computing concepts.

4.4 Spark Core

Spark Core 3.5.6 provides the foundation for RDD-based processing.

The project uses Spark Core for:

- Creating RDDs
- Parsing transaction records
- Pair RDD operations
- `reduceByKey`
- Partition management
- repartition
- coalesce
- Accumulators
- Broadcast variables
- Caching

Account-level transaction frequency and geographic transaction counts are examples of operations implemented using RDD-based processing.

4.5 Spark SQL

Spark SQL 3.5.6 is used for structured transaction analysis.

Validated transaction records are converted into DataFrames, allowing the project to perform structured operations such as:

- `groupBy`
- `count`
- `sum`
- `avg`
- Account-level statistics
- Transaction-type statistics
- Location statistics

This demonstrates how Spark can move from low-level RDD processing to higher-level structured analytics.

4.6 Spark Streaming

Spark Streaming 3.5.6 is used to demonstrate continuous transaction processing.

The project uses file-based streaming to simulate incoming transaction batches. Stateful processing is implemented using `updateStateByKey()`, allowing account transaction counts to be maintained across multiple micro-batches.

A separate sliding-window processor evaluates recent transaction activity using a **15-second window and 5-second slide interval**.

4.7 Java Runtime Environment

The project runs using **OpenJDK 17.0.20**.

Java provides the runtime environment required by both Scala and Apache Spark. The development environment was configured so that Spark could execute correctly with Java 17.

4.8 Build and Dependency Management

The project uses **sbt 2.0.7** as its build and package management tool.

The Spark dependencies used in the project are:

`org.apache.spark`

`spark-core`

3.5.6

org.apache.spark

spark-sql

3.5.6

org.apache.spark

spark-streaming

3.5.6

These are the actual Spark modules required by the implementation.

4.9 Development and Deployment Tools

The project was developed in a **Linux/WSL2 environment** and maintained using Git and GitHub.

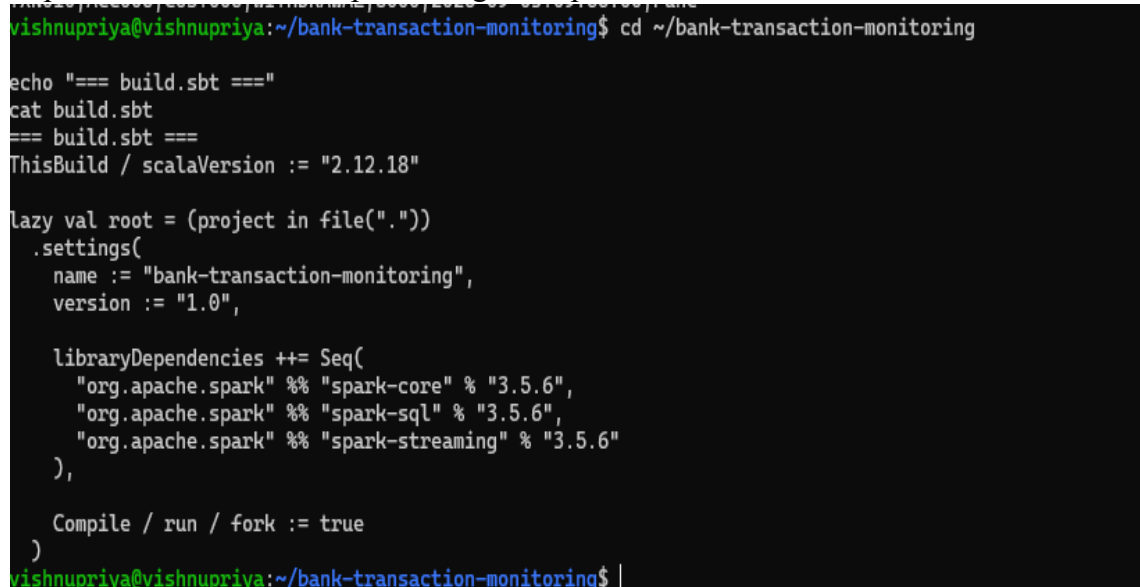
GitHub Actions is used to automatically validate the project build, while the monitoring dashboard is published using GitHub Pages. This provides a basic development-to-validation workflow for the Spark application.

4.10 Technology Selection Summary

The technology stack was selected to demonstrate a complete Spark-based data engineering workflow:

Scala → Apache Spark → RDD/Pair RDD → Streaming → DataFrames → Monitoring Rules → Dashboard → GitHub Actions

This combination allows the project to demonstrate both practical transaction monitoring and important distributed data-processing concepts.



```
vishnupriya@vishnupriya:~/bank-transaction-monitoring$ cd ~/bank-transaction-monitoring
echo "=== build.sbt ==="
cat build.sbt
=== build.sbt ===
ThisBuild / scalaVersion := "2.12.18"

lazy val root = (project in file("."))
  .settings(
    name := "bank-transaction-monitoring",
    version := "1.0",

    libraryDependencies ++= Seq(
      "org.apache.spark" %% "spark-core" % "3.5.6",
      "org.apache.spark" %% "spark-sql" % "3.5.6",
      "org.apache.spark" %% "spark-streaming" % "3.5.6"
    ),

    Compile / run / fork := true
  )
vishnupriya@vishnupriya:~/bank-transaction-monitoring$ |
```

Figure 3. Project build environment and GitHub Actions configuration showing Java 17 and sbt setup.

setup.

5. SYSTEM ARCHITECTURE AND PROCESSING FLOW

5.1 System Architecture

The Bank Transaction Monitoring system follows a layered processing architecture. Each layer performs a specific responsibility, starting from transaction ingestion and ending with monitoring results and dashboard presentation.

The major layers are:

1. **Input Layer** – Receives CSV transaction records and streaming files.
2. **Parsing Layer** – Safely parses input records into the BankTransaction case class.
3. **RDD Processing Layer** – Performs Pair RDD account and location aggregations.
4. **Monitoring Layer** – Applies frequency, amount, and geographic thresholds.
5. **Streaming Layer** – Processes incoming data using Spark Streaming and `updateStateByKey()`.
6. **Window Layer** – Performs recent-activity analysis using a 15-second window and 5-second slide.
7. **Structured Analytics Layer** – Uses DataFrames and `groupBy` aggregations.
8. **Presentation Layer** – Displays monitoring results through the HTML/CSS dashboard.
9. **Validation Layer** – Stores execution evidence and validates the application using `spark-submit` and GitHub Actions.

5.2 Processing Flow

The complete processing flow of the system is:

Input CSV → Safe Parser → Cached RDD → Pair RDD / DataFrame Analytics → Monitoring Rules → Streaming / Window Paths → Final Summary → Dashboard and Saved Execution Evidence

The input transaction records are first validated and converted into structured transaction objects. The validated data is then processed through RDD-based and DataFrame-based operations.

The monitoring layer evaluates the transaction data against predefined thresholds. Streaming and window-processing paths provide additional analysis of ongoing and recent transaction activity. Finally, the results are consolidated and presented through the monitoring dashboard.

5.3 Architecture Components

Input and Parsing

The system accepts transaction records in CSV format. A safe parsing mechanism validates each record before creating a BankTransaction object.

Distributed Processing

Spark RDDs provide distributed processing of the transaction records. Pair RDD operations are used for account-frequency and location-based aggregation.

Monitoring Rules

Three primary monitoring rules are applied:

- Account frequency greater than 3
- Transaction amount greater than ₹50,000
- Location transaction count greater than 3

Streaming and Window Processing

The streaming path maintains account-level state across micro-batches. The window path evaluates recent activity using the configured sliding window.

Structured Analytics

Spark DataFrames are used to generate account-level, transaction-type, and geographic statistics.

Presentation and Validation

The final results are presented through the HTML/CSS dashboard, while saved execution outputs, spark-submit, and GitHub Actions provide validation evidence.



Figure 4. System Architecture of the Bank Transaction Monitoring System.

6. CORE DATA MODEL

6.1 Transaction Data Schema

The Bank Transaction Monitoring system uses a structured transaction schema to represent individual banking events. Each transaction contains identification, customer, transaction, financial, temporal, and geographical information. The schema is implemented in Scala using the BankTransaction case class and is shared by the different processing modules.

The transaction dataset contains seven fields:

Field	Data Type	Description
transactionId	String	Unique identifier of the transaction
accountId	String	Identifier of the bank account
customerId	String	Identifier of the customer
transactionType	String	Type of transaction such as DEPOSIT, WITHDRAWAL or TRANSFER
amount	Double	Monetary value of the transaction
timestamp	String	Date and time associated with the transaction
location	String	Geographic location of the transaction

Table 2. Transaction Data Schema

The accountId field is particularly important because it is used as the key for Pair RDD account-level aggregation. The amount field is used for high-value transaction detection, while location is used for geographic monitoring.

6.2 BankTransaction Case Class

The transaction schema is implemented through a Scala case class named BankTransaction. Case classes provide a convenient structure for representing immutable records and are directly suitable for Spark RDD and DataFrame processing.

The implementation is defined in Models.scala and contains all seven transaction attributes:

```
case class BankTransaction(  
  transactionId: String,  
  accountId: String,  
  customerId: String,  
  transactionType: String,
```

```
amount: Double,  
timestamp: String,  
location: String  
)
```

The same data model is reused by the batch, streaming, window, and final monitoring processors. This provides consistency across the different processing paths.

```
vishnupriya@vishnupriya:~/bank-transaction-monitoring$ cd ~/bank-transaction-monitoring  
  
echo "=== BankTransaction Data Model ==="  
cat src/main/scala/Models.scala  
=== BankTransaction Data Model ===  
case class BankTransaction(  
  transactionId: String,  
  accountId: String,  
  customerId: String,  
  transactionType: String,  
  amount: Double,  
  timestamp: String,  
  location: String  
)  
vishnupriya@vishnupriya:~/bank-transaction-monitoring$ |
```

Figure 5. BankTransaction case class defining the transaction data model.

6.3 Sample Transaction Dataset

The project uses a CSV-based transaction dataset as the primary input. The dataset contains transaction identifiers, account identifiers, customer identifiers, transaction types, transaction amounts, timestamps, and locations.

The sample execution contains **20 valid transactions**. The dataset represents transactions distributed across multiple accounts and geographic locations, allowing the monitoring system to demonstrate account-frequency, amount-based, and geographic analysis.

The sample data contains different transaction types, including deposits, withdrawals, and transfers. The amount and location fields provide the basis for the monitoring rules implemented in the project.

```
vishnupriya@vishnupriya:~/bank-transaction-monitoring$ cd ~/bank-transaction-monitoring  
  
echo "=== Transaction Dataset ==="  
head -11 data/input/transactions.csv  
=== Transaction Dataset ===  
transactionId,accountId,customerId,transactionType,amount,timestamp,location  
TXN001,ACC001,CUST001,DEPOSIT,50000,2026-09-03T09:00:00,Hyderabad  
TXN002,ACC001,CUST001,WITHDRAWAL,5000,2026-09-03T09:15:00,Hyderabad  
TXN003,ACC002,CUST002,TRANSFER,2500,2026-09-03T09:20:00,Chennai  
TXN004,ACC003,CUST003,WITHDRAWAL,15000,2026-09-03T09:25:00,Bangalore  
TXN005,ACC004,CUST004,TRANSFER,75000,2026-09-03T09:30:00,Mumbai  
TXN006,ACC001,CUST001,TRANSFER,3000,2026-09-03T09:35:00,Hyderabad  
TXN007,ACC002,CUST002,WITHDRAWAL,4500,2026-09-03T09:40:00,Chennai  
TXN008,ACC005,CUST005,DEPOSIT,10000,2026-09-03T09:45:00,Delhi  
TXN009,ACC003,CUST003,TRANSFER,60000,2026-09-03T09:50:00,Bangalore  
TXN010,ACC006,CUST006,WITHDRAWAL,8000,2026-09-03T09:55:00,Pune  
vishnupriya@vishnupriya:~/bank-transaction-monitoring$ |
```

Figure 6. Sample transaction records from the Bank Transaction Monitoring input dataset.

6.4 Data Validation

Before transactions enter the monitoring calculations, the input records are validated by the parser. Header handling prevents the CSV schema row from being processed as a transaction. The parser verifies the expected field structure and converts the amount field into a numeric value.

Malformed records are not silently treated as valid transactions. Instead, they are counted using a Spark accumulator so that data-quality information can be reported separately from business-monitoring results.

For the validated sample:

- **Valid transactions:** 20
- **Malformed transactions:** 0
- **Accounts:** 10
- **Locations:** 7

This validation process ensures that downstream monitoring calculations operate only on successfully parsed transaction records.

7. BATCH PROCESSING MODULE

7.1 Batch Processing Workflow

The Batch Processing Module provides the primary offline processing path for the Bank Transaction Monitoring system. It reads transaction records from the CSV input dataset, validates and converts them into BankTransaction objects, and performs Spark-based analytical operations.

The batch workflow can be summarized as:

CSV Transaction Dataset



Header Removal



Safe CSV Parsing



BankTransaction Objects



RDD Processing



Pair RDD Aggregation



Monitoring Rules



DataFrame Analytics



Final Results

The batch processor demonstrates the use of Spark Core RDD operations together with DataFrame-based analytical processing.

7.2 Safe CSV Parsing

The input CSV file is processed using a safe parsing approach. The header row is removed before transaction records are converted into the application data model.

Each valid row is converted into a BankTransaction object. The parser extracts the transaction identifier, account identifier, customer identifier, transaction type, amount, timestamp, and location.

If a record does not satisfy the expected structure or contains an invalid numeric amount, it is treated as malformed. Such records are counted through the malformed transaction accumulator.

This approach prevents invalid input from entering subsequent monitoring calculations.

7.3 Malformed Transaction Handling

Data quality is an important part of the batch pipeline. Instead of terminating the complete processing job when an individual record is malformed, the implementation records the malformed transaction count.

The accumulator provides a centralized counter that can be inspected after processing. In the validated execution, the accumulator reported:

Total valid transactions: 20

Malformed transactions: 0

Therefore, all 20 records in the sample dataset were successfully parsed and processed.

7.4 Cached RDD Processing

After parsing, the transaction RDD can be cached so that the same dataset can be reused by multiple analytical operations without repeatedly rebuilding the RDD lineage.

The cached data is used for operations including:

- Account-level transaction counting
- High-amount transaction identification
- Geographic transaction analysis
- DataFrame conversion
- Additional monitoring calculations

Caching is useful in this project because the same transaction dataset participates in multiple downstream operations.

7.5 Batch Monitoring and Aggregation

The batch processor performs several monitoring operations.

Account Frequency Monitoring

Transactions are converted into Pair RDD records using the account identifier as the key:

(accountId, 1)

The records are then aggregated using `reduceByKey()` to obtain the transaction count for each account.

The configured frequency threshold is:

Frequency threshold = 3

An account with more than three transactions is classified as a high-frequency account.

In the sample execution, ACC001 recorded **6 transactions** and therefore generated a high-frequency alert.

High-Amount Monitoring

The amount threshold is:

Amount threshold = ₹50,000

Transactions exceeding this threshold are reported as high-amount transactions.

The sample execution identified four high-amount transactions:

- TXN005 — ₹75,000
- TXN009 — ₹60,000
- TXN014 — ₹90,000
- TXN015 — ₹120,000

Geographic Monitoring

Transaction records are also converted into a location-based Pair RDD:

(location, 1)

reduceByKey() is then used to calculate the number of transactions associated with each location.

The geographic frequency threshold is:

Geographic threshold = 3

Hyderabad recorded **7 transactions**, making it the high-frequency geographic location in the sample.

7.6 Partition Management

The batch processor also demonstrates Spark partition management.

The initial transaction RDD contains:

Transactions RDD partitions: 2

The project then demonstrates increasing the number of partitions using:

repartition(4)

which produces:

Partitions after repartition(4): 4

The number of partitions is subsequently reduced using:

coalesce(2)

resulting in:

Partitions after coalesce(2): 2

This demonstrates the practical difference between increasing partitions through repartitioning and reducing partitions through coalescing.

7.7 DataFrame Analytics

After the RDD-based monitoring operations, transaction records are converted into a Spark DataFrame.

The DataFrame provides a structured representation of the transaction dataset and enables SQL-style aggregation operations.

The project calculates account-level statistics including:

- Transaction count
- Total transaction amount
- Average transaction amount
- Maximum transaction amount

The DataFrame schema contains the same seven transaction fields defined by the BankTransaction model.

7.8 Batch Execution Results

The complete batch processor was successfully executed using Apache Spark. The validated execution produced:

Metric	Result
Valid transactions	20
Malformed transactions	0
High-frequency accounts	1
High-amount transactions	4
High-frequency locations	1
Initial partitions	2
After repartition	4
After coalesce	2

Table 3. Batch Execution Results

The execution therefore demonstrates the complete batch monitoring pipeline, from input validation and RDD processing through monitoring rules, partition management, and DataFrame analytics.

```

=====
BANK TRANSACTION MONITORING
=====

Total valid transactions: 20
Malformed transaction count:
-----
Malformed transactions: 0

First 5 valid transactions:
-----
BankTransaction(TXN001,ACC001,CUST001,DEPOSIT,50000.0,2026-09-03T09:00:00,Hyderabad)
BankTransaction(TXN002,ACC001,CUST001,WITHDRAWAL,5000.0,2026-09-03T09:15:00,Hyderabad)
BankTransaction(TXN003,ACC002,CUST002,TRANSFER,2500.0,2026-09-03T09:20:00,Chennai)
BankTransaction(TXN004,ACC003,CUST003,WITHDRAWAL,15000.0,2026-09-03T09:25:00,Bangalore)
BankTransaction(TXN005,ACC004,CUST004,TRANSFER,75000.0,2026-09-03T09:30:00,Mumbai)

Broadcast monitoring thresholds:
-----
Frequency threshold: 3
Amount threshold: ₹50000.00
Geographic threshold: 3

Partition information:
-----
Transactions RDD partitions: 2
Account Pair RDD partitions: 2
After reduceByKey partitions: 2

Transaction count by account:
-----
ACC007 -> 1
ACC001 -> 6
ACC010 -> 1
ACC002 -> 3
ACC003 -> 3
ACC009 -> 1
ACC005 -> 1
ACC004 -> 2
ACC008 -> 1
ACC006 -> 1

High transaction frequency accounts:
-----
ALERT -> ACC001 has 6 transactions

Number of high-frequency accounts: 1

High amount transactions:
-----
ALERT -> TXN014 | Account: ACC007 | Amount: ₹90000.00 | Type: WITHDRAWAL | Location: Delhi
ALERT -> TXN015 | Account: ACC008 | Amount: ₹120000.00 | Type: TRANSFER | Location: Kolkata
ALERT -> TXN005 | Account: ACC004 | Amount: ₹75000.00 | Type: TRANSFER | Location: Mumbai
ALERT -> TXN009 | Account: ACC003 | Amount: ₹60000.00 | Type: TRANSFER | Location: Bangalore

Number of high-amount transactions: 4

```

Figure 7. Batch Processing Module Execution Output

```

Transaction count by location:
-----
Chennai -> 3
Delhi -> 2
Hyderabad -> 7
Kolkata -> 1
Bangalore -> 3
Mumbai -> 3

High-frequency geographic locations:
-----
ALERT -> Hyderabad has 7 transactions

Number of high-frequency locations: 1

After repartition:
-----
Partitions after repartition(4): 4

After coalesce:
-----
Partitions after coalesce(2): 2

Transaction DataFrame:
-----
+-----+-----+-----+-----+-----+-----+-----+
|transactionId|accountId|customerId|transactionType|amount|timestamp|location|
+-----+-----+-----+-----+-----+-----+-----+
|TXN001|ACC001|CUST001|DEPOSIT|50000.0|2026-09-03T09:00:00|Hyderabad|
|TXN002|ACC001|CUST001|WITHDRAWAL|5000.0|2026-09-03T09:15:00|Hyderabad|
|TXN003|ACC002|CUST002|TRANSFER|2500.0|2026-09-03T09:20:00|Chennai|
|TXN004|ACC003|CUST003|WITHDRAWAL|15000.0|2026-09-03T09:25:00|Bangalore|
|TXN005|ACC004|CUST004|TRANSFER|75000.0|2026-09-03T09:30:00|Mumbai|
+-----+-----+-----+-----+-----+-----+-----+

only showing top 5 rows

DataFrame schema:
-----
root
  |-- transactionId: string (nullable = true)
  |-- accountId: string (nullable = true)
  |-- customerId: string (nullable = true)
  |-- transactionType: string (nullable = true)
  |-- amount: double (nullable = false)
  |-- timestamp: string (nullable = true)
  |-- location: string (nullable = true)

Account transaction statistics:
-----
+-----+-----+-----+-----+-----+
|accountId|transactionCount|totalAmount|averageAmount|maximumAmount|
+-----+-----+-----+-----+-----+
|ACC001|6|65200.0|10866.666666666666|50000.0|
|ACC002|3|38500.0|12833.333333333333|15000.0|
|ACC003|3|80500.0|26833.333333333332|60000.0|
|ACC004|2|85000.0|42500.0|75000.0|
|ACC005|1|10000.0|10000.0|10000.0|
|ACC006|1|2000.0|2000.0|2000.0|
|ACC007|1|90000.0|90000.0|90000.0|
|ACC008|1|120000.0|120000.0|120000.0|
|ACC009|1|7000.0|7000.0|7000.0|
|ACC010|1|25000.0|25000.0|25000.0|
+-----+-----+-----+-----+-----+

```

8. ACCOUNT FREQUENCY MONITORING

8.1 Account Frequency Monitoring Overview

Account frequency monitoring is used to identify accounts that perform an unusually large number of transactions within the analyzed dataset. In the Bank Transaction Monitoring system, transaction frequency is calculated using Spark **Pair RDD** operations.

Each valid transaction is converted into a key-value pair where the accountId acts as the key and the value is 1.

(accountId, 1)

For example:

(ACC001, 1)

(ACC001, 1)

(ACC002, 1)

(ACC003, 1)

These records are aggregated using `reduceByKey()` to calculate the total number of transactions associated with each account.

8.2 Pair RDD Account Aggregation

The Pair RDD approach provides an efficient method for performing account-level aggregation.

The processing sequence is:

BankTransaction

↓

Extract accountId

↓

(accountId, 1)

↓

`reduceByKey()`

↓

Account Transaction Count

The `reduceByKey()` operation combines values having the same account identifier. As a result, the system obtains a transaction count for every account present in the dataset.

For example, if ACC001 occurs six times, the aggregation produces:

ACC001 → 6

This account-level result is then used by the monitoring rule.

8.3 Frequency Threshold Rule

The system uses a configurable frequency threshold for identifying high-frequency accounts.

The configured threshold is:

Frequency Threshold = 3

The monitoring condition is:

Transaction Count > 3

↓

High-Frequency Account

Therefore, accounts with more than three transactions are reported as high-frequency accounts.

The threshold is distributed using a **Spark broadcast variable**, allowing the monitoring logic to access the configuration efficiently across Spark tasks.

8.4 High-Frequency Account Detection

The sample dataset contains transactions distributed across ten accounts. After Pair RDD aggregation, the account transaction counts are evaluated against the frequency threshold.

The important result is:

ACC001 → 6

Since six transactions are greater than the configured threshold of three, ACC001 is identified as a high-frequency account.

The execution produced the alert:

ALERT → ACC001 has 6 transactions

The total number of high-frequency accounts detected in the sample dataset is:

1

8.5 Monitoring Result

The account-frequency monitoring process successfully demonstrates how Spark can transform individual transaction records into account-level monitoring information.

The complete process consists of:

1. Reading valid transaction records.
2. Extracting the accountId.
3. Creating (accountId, 1) Pair RDD records.

4. Applying `reduceByKey()` to calculate transaction counts.
5. Comparing the counts with the broadcast frequency threshold.
6. Generating an alert for accounts exceeding the threshold.

This approach demonstrates the practical use of **Pair RDD aggregation**, **`reduceByKey()`**, **broadcast variables**, and **threshold-based monitoring** within the project.

```

BANK TRANSACTION MONITORING
=====

Total valid transactions: 20
Malformed transaction count:
=====
Malformed transactions: 0
First 5 valid transactions:
=====
BankTransaction(TXM001,ACC001,CUST001,DEPOSIT,50000.0,2026-09-03T09:00:00,Hyderabad)
BankTransaction(TXM002,ACC001,CUST001,WITHDRAWAL,5000.0,2026-09-03T09:15:00,Hyderabad)
BankTransaction(TXM003,ACC002,CUST002,TRANSFER,2500.0,2026-09-03T09:30:00,Chennai)
BankTransaction(TXM004,ACC003,CUST003,WITHDRAWAL,15000.8,2026-09-03T09:25:00,Bangalore)
BankTransaction(TXM005,ACC004,CUST004,TRANSFER,75000.8,2026-09-03T09:30:00,Mumbai)

Broadcast monitoring thresholds:
=====
Frequency threshold: 3
Amount threshold: ₹50000.00
Geographic threshold: 3

Partition information:
=====
Transactions RDD partitions: 2
Account Pair RDD partitions: 2
After reduceByKey partitions: 2

Transaction count by account:
=====
ACC007 -> 1
ACC001 -> 6
ACC018 -> 1
ACC002 -> 3
ACC003 -> 3
ACC009 -> 1
ACC004 -> 2
ACC008 -> 1
ACC005 -> 1
ACC006 -> 1

High transaction frequency accounts:
=====
ALERT -> ACC001 has 6 transactions

Number of high-frequency accounts: 1

High amount transactions:
=====
ALERT -> TXM014 | Account: ACC007 | Amount: ₹90000.00 | Type: WITHDRAWAL | Location: Delhi
ALERT -> TXM015 | Account: ACC008 | Amount: ₹120000.00 | Type: TRANSFER | Location: Kolkata
ALERT -> TXM005 | Account: ACC004 | Amount: ₹75000.00 | Type: TRANSFER | Location: Mumbai
ALERT -> TXM009 | Account: ACC003 | Amount: ₹60000.00 | Type: TRANSFER | Location: Bangalore

Number of High-amount transactions: 4

Transaction count by location:
=====
Kolkata -> 1
Bangalore -> 3
Mumbai -> 3
Chennai -> 3
Delhi -> 2
Hyderabad -> 7
Pune -> 1

```

Figure 8. Account-wise transaction frequency analysis and high-frequency account detection.

9. HIGH-AMOUNT AND GEOGRAPHIC MONITORING

9.1 High-Amount Transaction Monitoring

In addition to account-frequency analysis, the Bank Transaction Monitoring system identifies transactions whose monetary value exceeds a predefined threshold. This provides a simple rule-based mechanism for detecting potentially significant transactions that require further attention.

The amount threshold used by the system is:

Amount Threshold = ₹50,000

The monitoring rule can be represented as:

Transaction Amount > ₹50,000

↓

High-Amount Alert

Each valid BankTransaction record is evaluated against this threshold. Transactions satisfying the condition are reported as high-amount transactions.

9.2 High-Amount Transaction Detection

The batch processor applies the amount-based rule to the validated transaction dataset. The monitoring process examines the amount field of each transaction and generates an alert when the configured threshold is exceeded.

The sample execution identified four transactions above the threshold:

Transaction	Account	Amount	Type	Location
TXN005	ACC004	₹75,000	TRANSFER	Mumbai
TXN009	ACC003	₹60,000	TRANSFER	Bangalore
TXN014	ACC007	₹90,000	WITHDRAWAL	Delhi
TXN015	ACC008	₹120,000	TRANSFER	Kolkata

Table 4. High-Amount Transaction Results

Therefore, the total number of high-amount transactions detected is:

4

The result demonstrates how threshold-based filtering can be applied to transaction data using Spark.

9.3 Geographic Transaction Monitoring

The system also monitors transaction concentration by geographic location. This analysis is performed by extracting the location field from each transaction and creating a location-based Pair RDD.

The records follow the structure:

(location, 1)

The location records are then aggregated using `reduceByKey()` to calculate the number of transactions associated with each location.

The processing flow is:

BankTransaction



Extract location



(location, 1)



`reduceByKey()`



Location Transaction Count



Threshold Comparison



Geographic Alert

This enables the system to identify locations with unusually high transaction concentration.

9.4 Geographic Frequency Rule

The configured geographic threshold is:

Geographic Threshold = 3

The monitoring condition is:

Location Transaction Count > 3



High-Frequency Geographic Location

Locations with more than three transactions are therefore identified by the monitoring system.

The sample dataset produced the following important result:

Hyderabad → 7

Since seven transactions exceed the threshold of three, Hyderabad is classified as a high-frequency geographic location.

9.5 Geographic Monitoring Results

The location-level transaction counts obtained from the sample dataset are:

Location	Transaction Count
Hyderabad	7
Chennai	3
Bangalore	3
Mumbai	3
Delhi	2
Kolkata	1
Pune	1

Table 5. Geographic Monitoring Results

Only Hyderabad exceeds the configured geographic threshold.

The execution generated the alert:

ALERT → Hyderabad has 7 transactions

The total number of high-frequency geographic locations detected is:

1

9.6 Combined Monitoring Analysis

The high-amount and geographic monitoring functions complement account-frequency monitoring.

The system therefore evaluates three major monitoring dimensions:

1. **Account frequency** — identifies accounts with unusually high transaction counts.
2. **Transaction amount** — identifies transactions exceeding ₹50,000.
3. **Geographic frequency** — identifies locations with more than three transactions.

For the validated sample, the results were:

Monitoring Rule	Threshold	Result
Account frequency	> 3	1 account
Transaction amount	> ₹50,000	4 transactions
Geographic frequency	> 3	1 location

Table 6. Combined Monitoring Results

These results demonstrate that the Spark pipeline can apply multiple monitoring rules to the same transaction dataset and produce separate alert categories.

```

High-frequency geographic locations:
-----
ALERT -> Hyderabad has 7 transactions

Number of high-frequency locations: 1

After repartition:
-----
Partitions after repartition(4): 4

After coalesce:
-----
Partitions after coalesce(2): 2

Transaction DataFrame:
-----
+-----+-----+-----+-----+-----+-----+-----+
|transactionId|accountId|customerId|transactionType|amount|timestamp|location|
+-----+-----+-----+-----+-----+-----+-----+
|TXN001|ACC001|CUST001|DEPOSIT|50000.0|2026-09-03T09:00:00|Hyderabad|
|TXN002|ACC001|CUST001|WITHDRAWAL|5000.0|2026-09-03T09:15:00|Hyderabad|
|TXN003|ACC002|CUST002|TRANSFER|2500.0|2026-09-03T09:20:00|Chennai|
|TXN004|ACC003|CUST003|WITHDRAWAL|15000.0|2026-09-03T09:25:00|Bangalore|
|TXN005|ACC004|CUST004|TRANSFER|75000.0|2026-09-03T09:30:00|Mumbai|
+-----+-----+-----+-----+-----+-----+-----+

only showing top 5 rows

DataFrame schema:
-----
root
 |-- transactionId: string (nullable = true)
 |-- accountId: string (nullable = true)
 |-- customerId: string (nullable = true)
 |-- transactionType: string (nullable = true)
 |-- amount: double (nullable = false)
 |-- timestamp: string (nullable = true)
 |-- location: string (nullable = true)

Account transaction statistics:
-----
+-----+-----+-----+-----+-----+
|accountId|transactionCount|totalAmount|averageAmount|maximumAmount|
+-----+-----+-----+-----+-----+
|ACC001|6|65500.0|10916.666666666666|50000.0|
|ACC002|3|10500.0|3500.0|4500.0|
|ACC003|3|80500.0|26833.333333333332|60000.0|
|ACC004|2|85000.0|42500.0|75000.0|
|ACC005|1|10000.0|10000.0|10000.0|
|ACC006|1|8000.0|8000.0|8000.0|
|ACC007|1|90000.0|90000.0|90000.0|
|ACC008|1|120000.0|120000.0|120000.0|
|ACC009|1|7000.0|7000.0|7000.0|
|ACC010|1|25000.0|25000.0|25000.0|
+-----+-----+-----+-----+-----+

[success] elapsed time: 7 s
vikram@vikram:~/bank-transaction-monitoring$

```

Figure 9. Geographic transaction frequency analysis and high-frequency location detection.

10. STATEFUL STREAMING MODULE

10.1 Introduction to Streaming Processing

The Stateful Streaming Module extends the Bank Transaction Monitoring system from static batch analysis to continuous transaction processing. Instead of processing only a fixed CSV dataset, the streaming processor monitors incoming transaction files placed in a designated streaming input directory.

The implementation uses **Apache Spark Streaming** and processes incoming data using micro-batches. This allows the system to maintain account-level transaction state across multiple batches.

The streaming workflow is:

Incoming Transaction File



Spark Streaming Input



Micro-Batch



CSV Parsing



BankTransaction



Account-Level Pair RDD



updateStateByKey()



Updated Account State



Monitoring Output

10.2 File-Based Streaming

The project uses a directory-based streaming source for receiving transaction files.

New CSV files placed inside the streaming input directory are detected by Spark Streaming and processed as they arrive.

The streaming input directory is:

data/input/stream/

This approach provides a simple event-driven mechanism for demonstrating streaming transaction processing without requiring an external message broker.

10.3 Micro-Batch Processing

Spark Streaming divides incoming data into small time-based batches. In this project, the streaming processor uses a **5-second batch interval**.

Therefore, the incoming transaction records are processed approximately every five seconds as new streaming data becomes available.

Each micro-batch is processed independently while the account state is maintained between batches.

10.4 Stateful Account Tracking

A major feature of the streaming processor is stateful account tracking using `updateStateByKey()`.

For every incoming transaction, the account identifier is converted into a key-value pair:

(accountId, 1)

The state-update operation combines the current batch value with the previously maintained account state.

Conceptually:

Previous Account State

+

Current Batch Transactions

↓

`updateStateByKey()`

↓

Updated Account State

This allows the system to maintain cumulative transaction counts even when transactions arrive in different batches.

10.5 `updateStateByKey()`

The `updateStateByKey()` operation is used to maintain information across streaming batches.

For example, if the first batch contains:

ACC001 → 4

and a later batch contains:

ACC001 → 2

the maintained state can represent:

ACC001 → 6

This makes `updateStateByKey()` suitable for account-level monitoring where historical state must be retained while new transactions continue to arrive.

10.6 Checkpointing

The streaming processor uses checkpointing to preserve the information required for stateful processing.

The checkpoint directory used by the project is:

`data/checkpoint/`

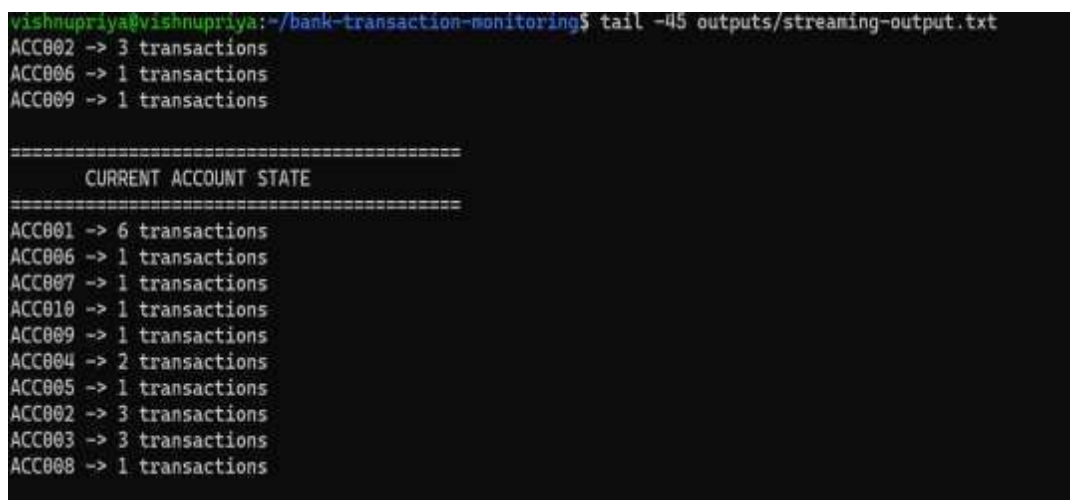
Checkpointing is particularly important for stateful streaming because the application needs to maintain state across micro-batches and recover processing information if the streaming application is interrupted.

10.7 Streaming Monitoring Results

The streaming execution was tested using transaction files placed into the streaming input directory. The processor successfully maintained account-level state and displayed the current transaction counts.

The streaming results demonstrate that:

- Incoming transaction files can be detected.
- Transactions are processed in micro-batches.
- Account-level state is maintained.
- `updateStateByKey()` updates cumulative account counts.
- Checkpointing is enabled for stateful processing.



```
vishnupriya@vishnupriya:~/bank-transaction-monitoring$ tail -45 outputs/streaming-output.txt
ACC002 -> 3 transactions
ACC006 -> 1 transactions
ACC009 -> 1 transactions

=====
CURRENT ACCOUNT STATE
=====
ACC001 -> 6 transactions
ACC006 -> 1 transactions
ACC007 -> 1 transactions
ACC010 -> 1 transactions
ACC009 -> 1 transactions
ACC004 -> 2 transactions
ACC005 -> 1 transactions
ACC002 -> 3 transactions
ACC003 -> 3 transactions
ACC008 -> 1 transactions
```

Figure 10. Stateful streaming execution showing cumulative account transaction state.

10.8 Streaming Processing Summary

The Stateful Streaming Module demonstrates how the batch-oriented transaction monitoring logic can be extended to continuously arriving transaction data.

The main technologies and concepts demonstrated are:

Component	Implementation
Streaming Source	File-based directory
Processing Model	Micro-batch
Batch Interval	5 seconds
Stateful Operation	updateStateByKey()
State	Account-level transaction count
Checkpointing	Enabled
Data Model	BankTransaction

This module provides the foundation for continuous account monitoring and prepares the system for more advanced real-time transaction analysis.

11. SLIDING WINDOW PROCESSING

11.1 Introduction to Window Processing

The Sliding Window Module is designed to analyze transaction activity within a limited and continuously moving time interval. While stateful streaming maintains cumulative account-level information, window processing focuses on **recent transaction activity**.

The project implements a sliding window with:

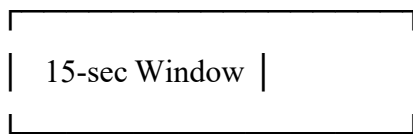
- **Window duration:** 15 seconds
- **Slide interval:** 5 seconds

The processing model can be represented as:

Incoming Transactions



5-second batches



Recent Transactions



Window Analysis



Monitoring Results

11.2 Window Duration

The window duration determines how much recent transaction data is included in each window.

The project uses:

Window Duration = 15 seconds

At any point in the streaming execution, the active window contains transaction records belonging to the most recent 15-second period.

This allows the system to analyze short-term transaction activity rather than relying only on cumulative historical counts.

11.3 Slide Interval

The slide interval determines how frequently the window moves forward and a new analysis is performed.

The project uses:

Slide Interval = 5 seconds

Therefore, the 15-second window is evaluated every five seconds.

Conceptually:

Time: 0----5----10----15----20 ----25

Window 1: [-----15 sec -----]

Window 2: [-----15 sec -----]

Window 3: [-----15 sec-----]

The overlapping nature of these windows allows recent transaction activity to be examined continuously.

11.4 Implementation

The sliding window is implemented using Spark Streaming's `window()` operation.

The processor applies:

```
window(  
    Seconds(15),  
    Seconds(5)  
)
```

The first parameter specifies the window duration, while the second parameter specifies the sliding interval.

This produces overlapping windows that continuously analyze recent transaction activity.

11.5 Recent Activity Analysis

Sliding-window processing is useful when transaction activity must be examined over a short period.

For example, several transactions occurring within a small time interval can be grouped into the same active window. The system can then perform account-level or transaction-level analysis on that recent subset.

The processing sequence is:

Streaming Transactions



Window(15 sec)



Slide(5 sec)



Recent Transaction Set



Aggregation



Window Results

This approach is different from cumulative stateful processing because the analysis is restricted to the current time window.

11.6 Short-Term Transaction Pattern Detection

The sliding-window approach provides a foundation for identifying short-term transaction bursts.

Instead of asking:

How many transactions has an account performed overall?

the window processor can answer:

How many transactions occurred within the recent 15-second period?

This distinction is useful for monitoring sudden changes in transaction activity.

The project therefore demonstrates both:

- **Stateful processing** for maintaining account-level state.
- **Sliding-window processing** for analyzing recent activity.

11.7 Window Processing Execution

The WindowProcessor reads incoming transaction data from the configured window input directory and applies the 15-second window with a 5-second slide.

The execution output provides evidence that the window-processing module starts successfully and processes the supplied transaction batches.

```

=====
WINDOW STREAMING STARTED
=====

Input directory:
data/input/window

Add CSV files into data/input/window/

Window = 15 seconds, Slide = 5 seconds

Press Ctrl+C to stop.

=====
15-SECOND TRANSACTION WINDOW
=====
ACC001 -> 6 transactions
ACC002 -> 3 transactions
ACC003 -> 3 transactions
ACC004 -> 2 transactions
ACC005 -> 1 transactions
ACC006 -> 1 transactions
ACC007 -> 1 transactions
ACC008 -> 1 transactions
ACC009 -> 1 transactions
ACC010 -> 1 transactions
=====
Window duration: 15 seconds
Sliding interval: 5 seconds

```

Figure 11. Sliding window processing execution using a 15-second window and 5-second slide interval.

11.8 Stateful Streaming vs Sliding Window

The two streaming approaches serve different analytical purposes.

Feature	Stateful Streaming	Sliding Window
Main purpose	Maintain cumulative state	Analyze recent activity
Main operation	updateStateByKey()	window()
Time behavior	State persists across batches	Data expires from the window
Project configuration	5-second micro-batch	15-second window / 5-second slide
Analysis	Account-level cumulative count	Recent transaction activity

Table 7. Stateful Streaming vs Sliding Window

Together, these components demonstrate how Spark Streaming can support both persistent state tracking and time-based transaction analysis.

12. FINAL MONITORING PROCESSOR

12.1 Overview

The FinalMonitoringProcessor integrates the major processing and monitoring concepts developed throughout the project into a single execution pipeline. It validates the transaction input, performs account-level and location-level analysis, applies monitoring thresholds, generates alerts, and produces structured DataFrame statistics.

The final processor provides a consolidated view of the Bank Transaction Monitoring system and serves as the main executable demonstration of the implemented Spark functionality.

12.2 Integrated Processing Workflow

The final monitoring pipeline follows the sequence below:

Transaction CSV



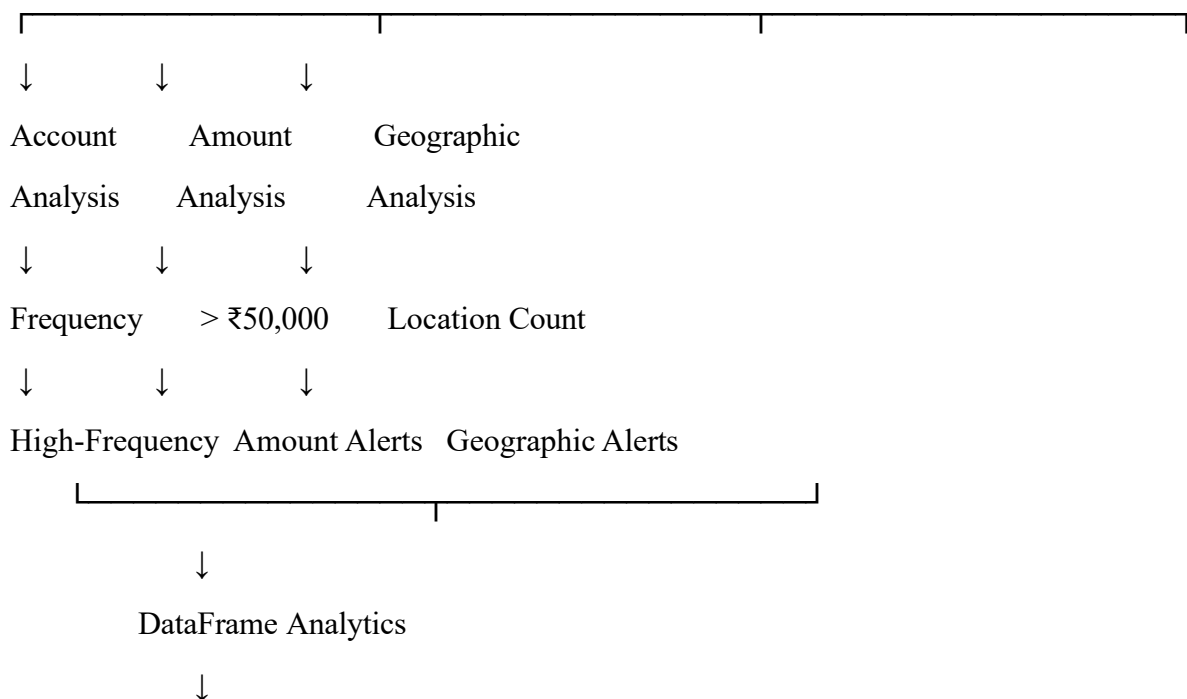
Safe CSV Parsing



Validation & Accumulator



Cached Transaction RDD



12.3 Configurable Monitoring Thresholds

The final processor uses three monitoring thresholds:

Monitoring Category	Threshold
Account frequency	> 3 transactions
Transaction amount	> ₹50,000
Geographic frequency	> 3 transactions

Table 8. Monitoring Thresholds

These thresholds determine when the system generates monitoring alerts.

Broadcast variables are used to distribute the threshold configuration efficiently across Spark tasks.

12.4 Integrated Monitoring Results

The validated execution processed **20 valid transactions** and recorded **0 malformed transactions**.

The final processor identified:

- **1 high-frequency account**
- **4 high-amount transactions**
- **1 high-frequency geographic location**

The high-frequency account was ACC001, with six transactions.

The four transactions exceeding the amount threshold were:

- TXN005 — ₹75,000
- TXN009 — ₹60,000
- TXN014 — ₹90,000
- TXN015 — ₹120,000

The geographic analysis identified Hyderabad as the high-frequency location, with seven transactions.

These results provide a consolidated demonstration of the monitoring rules implemented in the project.

12.5 DataFrame-Based Final Analytics

After the RDD-based monitoring operations, the transaction records are converted into a Spark DataFrame.

The final processor performs structured aggregations to calculate account-level statistics and transaction-level analytical information.

The DataFrame analysis includes:

- Transaction count by account
- Total transaction amount
- Average transaction amount
- Maximum transaction amount
- Transaction-type statistics
- Location statistics

This combination of RDD and DataFrame processing demonstrates how different Spark APIs can be used together within the same monitoring pipeline.

12.6 Partition Management in the Final Pipeline

The final processor also demonstrates Spark partition management.

The transaction RDD is initially processed with two partitions. The processor then demonstrates increasing the partition count using `repartition(4)` and reducing it again using `coalesce(2)`.

The observed sequence is:

Initial : 2 partitions

`repartition(4)` : 4 partitions

`coalesce(2)` : 2 partitions

This provides practical execution evidence for partition management within the Spark pipeline.

12.7 Final Execution Summary

The final execution produced the following summary:

Metric	Result
Valid transactions	20
Malformed transactions	0
High-frequency accounts	1

Metric	Result
High-amount transactions	4
High-frequency locations	1
Initial partitions	2
Repartitioned partitions	4
Final partitions	2

Table 9. Final Execution Summary

The successful execution confirms that the integrated monitoring pipeline can process the transaction dataset and generate results across multiple analytical dimensions.

12.8 Execution Evidence

The final processor was executed using spark-submit, providing an additional validation path beyond the sbt runMain execution.

```
vishnupriya@vishnupriya:~/bank-transaction-monitoring$ tail -30 outputs/spark-submit-output.txt
|location|transactionCount|totalAmount|
|-----|-----|-----|
|Bangalore|3|80500.0|
|Chennai|3|10500.0|
|Delhi|2|100000.0|
|Hyderabad|7|72500.0|
|Kolkata|1|120000.0|
|Mumbai|3|110000.0|
|Pune|1|8000.0|
|-----|-----|-----|

=====
PARTITION INFORMATION
=====
Original partitions: 2
After repartition(4): 4
After coalesce(2): 2

=====
MONITORING SUMMARY
=====
Valid transactions: 20
Malformed transactions: 0
High-frequency accounts: 1
High-amount transactions: 4
High-frequency locations: 1
Final monitoring pipeline completed.
```

Figure 12. Final Monitoring Processor execution using Spark Submit.

12.8 Execution Evidence

after the paragraph explaining the successful spark-submit execution.

The final processor demonstrates the complete integration of transaction validation, monitoring rules, DataFrame analytics, and partition management in the project.

13. TESTING AND VALIDATION

13.1 Testing Overview

Testing was performed to verify that the Bank Transaction Monitoring system correctly processes transaction data and produces the expected monitoring results. The validation covers input parsing, batch processing, streaming processing, sliding-window processing, final Spark execution, and CI validation.

The main objectives of testing were to verify:

- Correct transaction parsing
- Correct handling of malformed records
- Correct account-frequency calculation
- Correct high-amount detection
- Correct geographic analysis
- Correct stateful streaming behavior
- Correct sliding-window configuration
- Correct partition management
- Successful DataFrame aggregation
- Successful spark-submit execution
- Successful build and packaging through GitHub Actions

13.2 Input Validation Testing

The input-validation test verifies that the CSV file is correctly interpreted and that the header does not enter the transaction dataset.

The validated sample produced:

Test Item	Expected	Observed
Valid transactions	20	20
Malformed transactions	0	0
Accounts	10	10
Locations	7	7

Table 10. Input Validation Results

The result confirms that the supplied sample dataset was successfully parsed.

13.3 Batch Processing Validation

The BatchProcessor was executed to verify the complete batch-processing workflow.

The test confirmed:

- Transaction records were successfully loaded.
- Pair RDD account aggregation worked correctly.
- High-frequency accounts were identified.
- High-amount transactions were detected.
- Geographic transaction counts were calculated.
- DataFrame analytics executed successfully.
- Partition changes were successfully demonstrated.

The batch execution produced one high-frequency account, four high-amount transactions, and one high-frequency location.

13.4 Streaming Validation

The Stateful Streaming Module was tested using transaction files placed into the streaming input directory.

The test verified that Spark Streaming could:

1. Detect incoming transaction files.
2. Process them as micro-batches.
3. Maintain account-level state.
4. Update cumulative transaction counts using `updateStateByKey()`.
5. Continue processing using checkpoint support.

The streaming execution therefore validated the stateful processing component of the project.

13.5 Sliding Window Validation

The WindowProcessor was tested using the configured time-based window.

The implementation uses:

- **Window duration:** 15 seconds
- **Slide interval:** 5 seconds

The test verified that the streaming data could be processed using overlapping time windows for recent transaction analysis.

13.6 Final Spark Submit Validation

The integrated monitoring pipeline was also executed using spark-submit.

The final execution successfully produced the monitoring summary:

Validation Metric	Result
Valid transactions	20
Malformed transactions	0
High-frequency accounts	1
High-amount transactions	4
High-frequency locations	1

This confirms that the integrated Spark application executes successfully outside the development-oriented sbt runMain workflow.

13.7 Continuous Integration Validation

The project includes a GitHub Actions workflow for automated build validation.

The CI workflow performs separate stages for:

Clean

↓

Compile

↓

Package

The workflow uses Java 17 and the project's Scala/Spark build configuration.

Successful CI execution provides additional evidence that the project can be compiled and packaged in an automated environment.

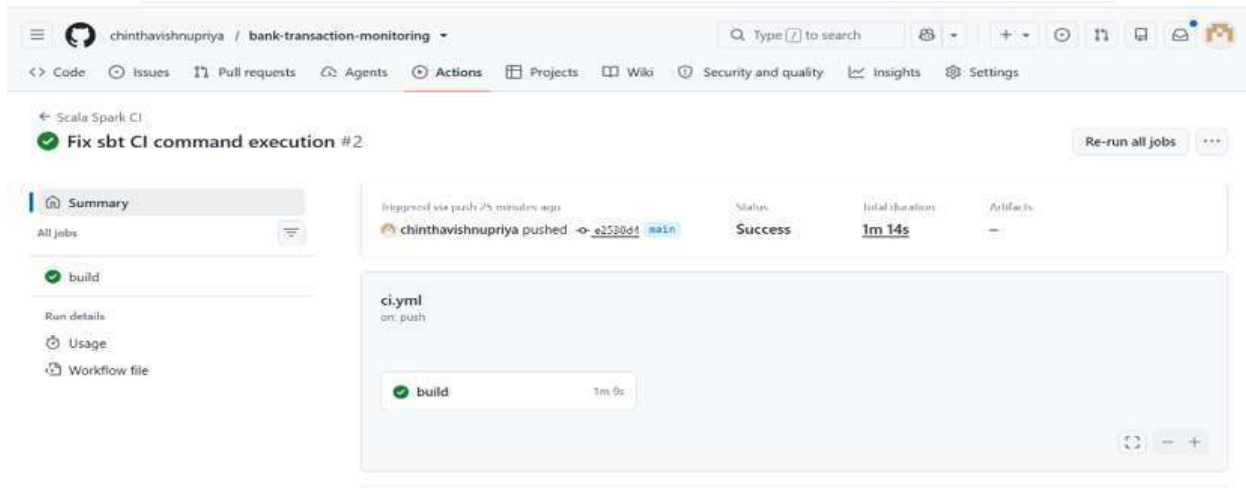


Figure 13. Successful GitHub Actions build and packaging validation.

13.8 Testing Summary

The testing process validates the major functional components of the project.

Component	Status
CSV Parsing	Successful
Data Validation	Successful
Pair RDD Aggregation	Successful
Frequency Monitoring	Successful
Amount Monitoring	Successful
Geographic Monitoring	Successful
Stateful Streaming	Successful
Sliding Window	Successful
DataFrame Analytics	Successful
Spark Submit	Successful
GitHub Actions	Successful

The overall testing results indicate that the implemented Bank Transaction Monitoring pipeline operates correctly for the supplied dataset and execution environment.

14. Pair RDD, Narrow and Wide Transformations

14.1 Introduction

Apache Spark provides Resilient Distributed Datasets (RDDs) as a fundamental abstraction for distributed data processing. In the Bank Transaction Monitoring project, RDDs are used to parse transaction records, perform account-level aggregations, identify high-frequency accounts, analyze transaction locations, and support partition management.

A particularly important part of the implementation is the use of **Pair RDDs**. A Pair RDD stores data in key-value form and is well suited for operations such as `reduceByKey`, `mapValues`, and `groupByKey`.

The project also demonstrates the distinction between **narrow transformations** and **wide transformations**. Understanding this distinction is important because wide transformations can introduce a shuffle between Spark stages.

14.2 Pair RDD Representation

For account-frequency monitoring, each transaction is converted into a key-value pair:

(accountId, 1)

For example:

(ACC001, 1)

(ACC001, 1)

(ACC002, 1)

(ACC003, 1)

These pairs can then be aggregated using `reduceByKey`.

The conceptual processing is:

Transaction RDD

↓

(accountId, 1)

↓

`reduceByKey(_ + _)`

↓

(accountId, transactionCount)

For the project's dataset, this produces results such as:

ACC001 → 6

ACC002 → 3

ACC003 → 3

ACC004 → 2

The result is then compared against the configured frequency threshold.

14.3 reduceByKey for Account Monitoring

The project uses reduceByKey to calculate the number of transactions associated with each account.

Conceptually:

```
val accountCounts =  
  transactionsRDD  
    .map(tx => (tx.accountId, 1))  
    .reduceByKey(_ + _)
```

Here:

- tx.accountId becomes the key.
- 1 represents one transaction.
- reduceByKey(_ + _) adds the values belonging to the same account.

The resulting Pair RDD contains the transaction frequency for every account.

This approach is more appropriate for account-frequency aggregation than collecting all records to the driver because the aggregation can be performed across Spark partitions.

14.4 High-Frequency Account Detection

After calculating account transaction counts, the monitoring rule is applied.

The configured frequency threshold in this project is:

Frequency threshold > 3

Therefore, an account with more than three transactions is identified as a high-frequency account.

The project dataset produces:

ACC001 → 6 transactions

Since:

6 > 3

ACC001 is reported as a high-frequency account.

This demonstrates how Pair RDD aggregation can directly support transaction-monitoring rules.

```

Transaction count by account:
-----
ACC007 -> 1
ACC002 -> 3
ACC001 -> 6
ACC004 -> 2
ACC010 -> 1
ACC008 -> 1
ACC006 -> 1
ACC003 -> 3
ACC009 -> 1
ACC005 -> 1

High transaction frequency accounts:
-----
ALERT -> ACC001 has 6 transactions

Number of high-frequency accounts: 1

High amount transactions:
-----
ALERT -> TXN014 | Account: ACC007 | Amount: ₹90000.00 | Type: WITHDRAWAL | Location: Delhi
ALERT -> TXN005 | Account: ACC004 | Amount: ₹75000.00 | Type: TRANSFER | Location: Mumbai
ALERT -> TXN015 | Account: ACC008 | Amount: ₹120000.00 | Type: TRANSFER | Location: Kolkata
ALERT -> TXN009 | Account: ACC003 | Amount: ₹60000.00 | Type: TRANSFER | Location: Bangalore

Number of high-amount transactions: 4

```

Figure 14. Pair RDD account-frequency aggregation and high-frequency account detection.

14.5 Narrow Transformations

A **narrow transformation** is a transformation where each output partition depends on a small number of input partitions, typically one.

Narrow transformations generally do not require a shuffle of data between partitions.

Examples used or applicable in this project include:

map

The transaction records are transformed into another representation.

```
transactionsRDD.map(tx => (tx.accountId, 1))
```

Each input transaction can independently produce its output pair.

filter

Filtering can be used to retain transactions satisfying a condition.

For example:

```
transactionsRDD.filter(_.amount > 50000)
```

Each record can be evaluated independently.

mapValues

For a Pair RDD, mapValues transforms values while preserving keys.

These operations can generally execute within the existing partition structure.

14.6 Wide Transformations

A **wide transformation** occurs when an output partition depends on data from multiple input partitions.

Such transformations generally require a **shuffle**.

The most important example in this project is:

`reduceByKey(_ + _)`

Transactions belonging to the same account may initially exist in different partitions.

For example:

Partition 1: (ACC001, 1)

Partition 2: (ACC001, 1)

Partition 3: (ACC001, 1)

Partition 4: (ACC001, 1)

Spark must bring the values associated with the same key together so that they can be reduced:

`ACC001` $\rightarrow 1 + 1 + 1 + 1 + \dots$

Therefore, `reduceByKey` is a wide transformation and can introduce a shuffle boundary.

14.7 Narrow vs Wide Transformations in the Project

Transformation	Category	Purpose
<code>map</code>	Narrow	Convert transaction records
<code>filter</code>	Narrow	Select records satisfying conditions
<code>mapValues</code>	Narrow	Transform Pair RDD values
<code>reduceByKey</code>	Wide	Aggregate values by account/location
<code>repartition</code>	Wide	Redistribute records across partitions
<code>coalesce</code>	Generally narrow	Reduce partition count without a full shuffle by default

Table 11. Narrow vs Wide Transformations in the Project

The distinction is important when analyzing Spark execution because wide transformations can create additional stages.

14.8 Why Pair RDDs Are Useful in This Project

Pair RDDs provide a convenient structure for monitoring operations based on keys.

Examples include:

Account-based monitoring

`accountId` $\rightarrow$ transaction count

Location-based monitoring

`location` $\rightarrow$ transaction count

This makes Pair RDDs particularly useful for identifying repeated activity associated with a specific account or geographic location.

14.9 Partition-Level Processing

Spark divides an RDD into partitions so that processing can be distributed across available executor cores.

In this project, partition management is demonstrated using:

`repartition(4)`

followed by:

`coalesce(2)`

14.10 Role in the Bank Transaction Monitoring Pipeline

The Pair RDD and transformation model can be summarized as:

CSV Transaction Records

↓

Safe CSV Parser

↓

Transaction RDD

↓

map / filter

↓

Pair RDD

(accountId, 1)

↓

reduceByKey

↓

Account Frequencies

↓

Frequency Threshold

↓

High-Frequency Alerts

The same principle is applied to geographic monitoring by grouping transactions according to their location.

15. DAG, Stages and Shuffle

15.1 Introduction

Apache Spark executes applications through a **Directed Acyclic Graph (DAG)** of operations. The transformations written in the Scala program are converted into a logical execution plan, which Spark divides into **stages** and **tasks**.

For the Bank Transaction Monitoring project, understanding the DAG is important because operations such as map, filter, reduceByKey, repartition, and coalesce have different execution characteristics.

15.2 Spark DAG

A DAG represents the sequence and dependency relationships between Spark transformations.

The simplified processing flow for the project is:

Transaction Input



Safe CSV Parsing



Transaction RDD



map / filter



Pair RDD



reduceByKey



Account / Location Aggregation



Monitoring Rules



Alerts and Results

Spark does not immediately execute every transformation when it is written. Transformations are evaluated lazily and execution begins when an action requires a result.

15.3 Lazy Evaluation

Spark transformations such as:

`map(...)`

`filter(...)`

`reduceByKey(...)`

build the computation plan without immediately executing the complete operation.

Execution occurs when an action is invoked, such as:

`count()`

`collect()`

`show()`

`saveAsTextFile()`

This lazy execution model allows Spark to analyze the computation and optimize its execution.

15.4 Stages

Spark divides a job into stages based largely on **shuffle boundaries**.

A stage can contain multiple operations that can be executed without redistributing data between partitions.

can generally remain within the same stage because these operations are narrow transformations.

When a shuffle-producing operation such as `reduceByKey` is encountered, Spark creates a stage boundary.

15.5 Shuffle

A **shuffle** occurs when Spark needs to redistribute data across partitions.

In account-frequency monitoring, transactions for the same account may initially be located in different partitions.

For example:

Partition 1 → ACC001

Partition 2 → ACC002

Partition 3 → ACC001

Partition 4 → ACC003

15.6 `reduceByKey` as a Shuffle Boundary

The project's account-frequency calculation uses:

`reduceByKey(_ + _)`

This is a wide transformation because data associated with the same key may originate from multiple partitions.

The shuffle is therefore an important point when analyzing the project's Spark execution plan.

15.8 Tasks and Partitions

Spark divides each stage into tasks based on the number of partitions that need to be processed.

For example, if an RDD contains four partitions:

Partition 0

Partition 1

Partition 2

Partition 3

Spark can create corresponding tasks to process those partitions.

This allows processing to occur in parallel across available CPU resources.

The project explicitly demonstrates partition changes:

Initial → 2 partitions

repartition → 4 partitions

coalesce → 2 partitions

This provides practical evidence of Spark's partition-based execution model.

15.9 Impact of Shuffle on Performance

Shuffle operations can be more expensive than narrow transformations because data may need to be redistributed between partitions.

Shuffle can involve:

- Data movement between partitions
- Serialization and deserialization
- Network transfer in distributed deployments
- Additional disk I/O when required
- Additional stage boundaries

Therefore, efficient Spark applications attempt to avoid unnecessary shuffles while still performing the required aggregations.

In this project, `reduceByKey` is necessary because account and location monitoring require grouped aggregation.

16. Broadcast Variables and Accumulator

16.1 Introduction

Apache Spark provides **Broadcast Variables** and **Accumulators** for specific distributed-processing requirements. Both are used in the Bank Transaction Monitoring project to improve the handling of shared configuration data and execution-level counters.

- **Broadcast variables** distribute read-only reference data efficiently to worker nodes.
- **Accumulators** allow executors to contribute values to a shared counter that can be observed by the driver.

These features are particularly useful in monitoring applications where common threshold values must be available during distributed processing and malformed records need to be counted.

16.2 Broadcast Variables

The monitoring system uses three threshold values:

Monitoring Rule Threshold

Account frequency > 3

Transaction amount > ₹50,000

Location frequency > 3

Instead of repeatedly sending these values as part of task execution, Spark can distribute them using a broadcast variable.

16.3 Broadcast Implementation

The project creates a broadcast configuration containing the monitoring thresholds.

Conceptually:

```
val thresholds = spark.sparkContext.broadcast(  
  Map(  
    "frequency" -> 3,  
    "amount" -> 50000.0,  
    "location" -> 3  
  )  
)
```

Worker-side processing can then access the shared values through the broadcast variable.

This is useful because the threshold configuration is **read-only** during processing.

16.4 Advantages of Broadcasting

Broadcasting provides several benefits:

1. Reduces repeated transmission of the same configuration.
2. Makes shared read-only values available to worker tasks.
3. Can reduce unnecessary communication overhead.
4. Is useful when the shared data is relatively small.
5. Keeps monitoring configuration separate from individual transaction records.

For this project, threshold values are small, so broadcasting is appropriate.

16.5 Accumulator

The project also uses an accumulator to count malformed transaction records.

The accumulator concept is:

Valid Transactions

↓

Process normally

Malformed Transactions

↓

Accumulator += 1

↓

Driver obtains final count

The project initializes a malformed-transaction counter and increments it whenever a transaction cannot be parsed correctly.

16.6 Malformed Transaction Monitoring

The input parser validates transaction records before converting them into BankTransaction objects.

If a record is invalid, it is excluded from normal transaction processing and the accumulator is incremented.

This allows the application to report the number of malformed records after processing.

In the actual project execution:

Valid Transactions = 20

Malformed Transactions = 0

16.7 Broadcast and Accumulator Comparison

Feature	Broadcast Variable	Accumulator
Main purpose	Share read-only data	Collect counters/metrics
Direction	Driver → executors	Executors → driver
Example in project	Monitoring thresholds	Malformed transaction count
Worker modification	No	Adds values
Typical use	Configuration/reference data	Statistics and monitoring

Table 12. Broadcast and Accumulator Comparison

16.8 Resource Management

After processing is completed, the broadcast variable can be explicitly released.

The project follows this lifecycle:

Create Broadcast

↓

Use During Processing

↓

Complete Monitoring

↓

Destroy Broadcast

This is useful for avoiding unnecessary retention of broadcast data after it is no longer required.

17. Cache, Persist and Partition Management

17.1 Introduction

Efficient memory and partition management are important aspects of distributed Spark applications. The Bank Transaction Monitoring project demonstrates these concepts through **RDD caching, persistence, repartitioning, and coalescing**.

These techniques help control how transaction data is stored and distributed during processing.

17.2 Why Caching Is Useful

Without caching, Spark may need to recompute the preceding transformations whenever an RDD is required by another action.

With caching:

First computation



RDD stored in memory



Reuse cached RDD



Less recomputation

For this project, the parsed transaction RDD is a suitable candidate because multiple monitoring operations use the same transaction data.

17.3 Persisting RDDs

Spark also provides the `persist()` mechanism for controlling how an RDD is stored.

For example:

```
transactionsRDD.persist()
```

Caching can be considered a convenient form of persistence using Spark's default storage level.

Persistence provides more control when an application needs a different storage strategy.

RDD



`persist()`



Storage Level



Reuse across operations

17.5 Cache vs Persist

Feature	Cache	Persist
Purpose	Store an RDD for reuse	Store an RDD using a selected storage level
Configuration	Uses default caching level	Allows explicit storage level
Main benefit	Avoids repeated computation	Provides storage flexibility
Project relevance	Reused transaction RDD	Persistence concept demonstrated

The project uses caching for the transaction-processing workflow and discusses persistence as an important Spark optimization technique.

17.6 Unpersisting Data

Cached data should be released when it is no longer required.

The project therefore removes the cached RDD after processing:

```
transactionsRDD.unpersist()
```

The lifecycle is:

Create RDD

↓

Cache RDD

↓

Perform monitoring operations

↓

Complete processing

↓

Unpersist RDD

This prevents unnecessary memory retention.

17.12 Application of These Concepts

The Bank Transaction Monitoring pipeline performs multiple analyses over transaction records:

- Account-frequency monitoring
- High-amount detection
- Geographic monitoring
- DataFrame statistics
- Transaction-type analysis

Because the parsed transaction data can be reused by several operations, caching can reduce unnecessary recomputation.

Partition management then controls how this data is distributed across Spark tasks.

17.13 Execution Evidence

The project execution confirmed the partition sequence:

Initial partitions : 2

After repartition : 4

After coalesce : 2

This provides practical evidence that partition manipulation was executed as part of the monitoring pipeline.

The final Spark-submit execution also completed successfully with the configured partition operations.

18. DataFrame Analytics and Aggregations

18.1 DataFrame Conversion

The validated BankTransaction RDD is converted into a Spark DataFrame. The DataFrame provides a structured schema containing transaction ID, account ID, customer ID, transaction type, amount, timestamp, and location.

18.2 Account-Level Aggregation

The project uses `groupBy("accountId")` to calculate account-level transaction statistics, including transaction count, total amount, average amount, minimum amount, and maximum amount.

This provides a structured view of account activity and complements the Pair RDD frequency analysis.

18.3 Transaction-Type and Location Analysis

DataFrame aggregations are also applied to `transactionType` and `location`. This helps identify transaction distributions and frequently used locations.

The dataset contains **20 valid transactions**, with **Hyderabad having the highest location count of 7**, exceeding the configured geographic threshold of 3.

```
only showing top 5 rows

DataFrame schema:
-----
root
 |-- transactionId: string (nullable = true)
 |-- accountId: string (nullable = true)
 |-- customerId: string (nullable = true)
 |-- transactionType: string (nullable = true)
 |-- amount: double (nullable = false)
 |-- timestamp: string (nullable = true)
 |-- location: string (nullable = true)

Account transaction statistics:
-----
+-----+-----+-----+-----+-----+
|accountId|transactionCount|totalAmount|averageAmount|maximumAmount|
+-----+-----+-----+-----+-----+
|ACC001|6|65500.0|10916.666666666666|50000.0|
|ACC002|3|10500.0|3500.0|4500.0|
|ACC003|3|80500.0|26833.333333333332|60000.0|
|ACC004|2|85000.0|42500.0|75000.0|
|ACC005|1|10000.0|10000.0|10000.0|
|ACC006|1|8000.0|8000.0|8000.0|
|ACC007|1|90000.0|90000.0|90000.0|
|ACC008|1|120000.0|120000.0|120000.0|
|ACC009|1|7000.0|7000.0|7000.0|
|ACC010|1|25000.0|25000.0|25000.0|
+-----+-----+-----+-----+-----+

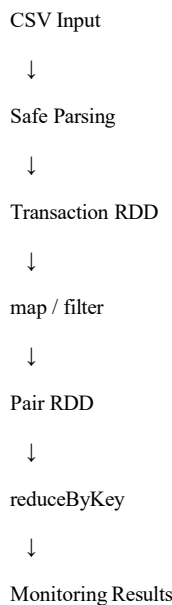
[success] elapsed time: 8 s
```

Figure 15: Spark DataFrame schema and account-level transaction statistics

19. RDD Lineage and Fault Tolerance

19.1 RDD Lineage

RDDs maintain information about how they were created through a sequence of transformations. This dependency information is called **lineage**.



19.2 Fault Tolerance

Spark uses RDD lineage to recover lost partitions. If a partition is lost because of an executor failure, Spark can recompute the required data using the previous transformations instead of requiring the entire dataset to be processed again.

This provides fault tolerance without manually maintaining backup copies of every intermediate RDD.

19.3 Lineage in the Project

The project contains multiple transformation stages, including parsing, filtering, Pair RDD conversion, aggregation, and DataFrame processing.

Therefore, Spark can track the dependencies between these processing steps and use them for recovery when required.

19.4 Importance

RDD lineage provides:

- **Fault tolerance**
- **Recovery of lost partitions**
- **Reliable distributed processing**
- **Recomputation of intermediate data**

19.5 Execution Evidence

The project was executed successfully using both the Scala/SBT workflow and spark-submit. The final execution processed **20 valid transactions** with **0 malformed transactions**.

The monitoring results included:

- **1 high-frequency account** — ACC001
- **4 high-amount transactions**
- **1 high-frequency location** — Hyderabad
- Partition sequence: **2 → 4 → 2**

19.6 Monitoring Dashboard

An HTML/CSS dashboard was developed to present the final monitoring results in a clear format.

The dashboard displays:

- Valid transactions
- Malformed transactions
- High-frequency accounts
- High-amount transactions
- High-frequency locations
- Monitoring thresholds
- Account-level statistics
- Geographic monitoring results
- Pipeline completion status

19.7 Dashboard Purpose

The dashboard provides a visual presentation layer for the Spark processing results. It converts the terminal-based execution results into an easier-to-read monitoring interface.

Spark Processing



Monitoring Results



Saved Output

HTML/CSS Dashboard



19.8 Final Dashboard Result

The completed dashboard confirms that the monitoring pipeline executed successfully and presents the important results from the 20-transaction dataset.

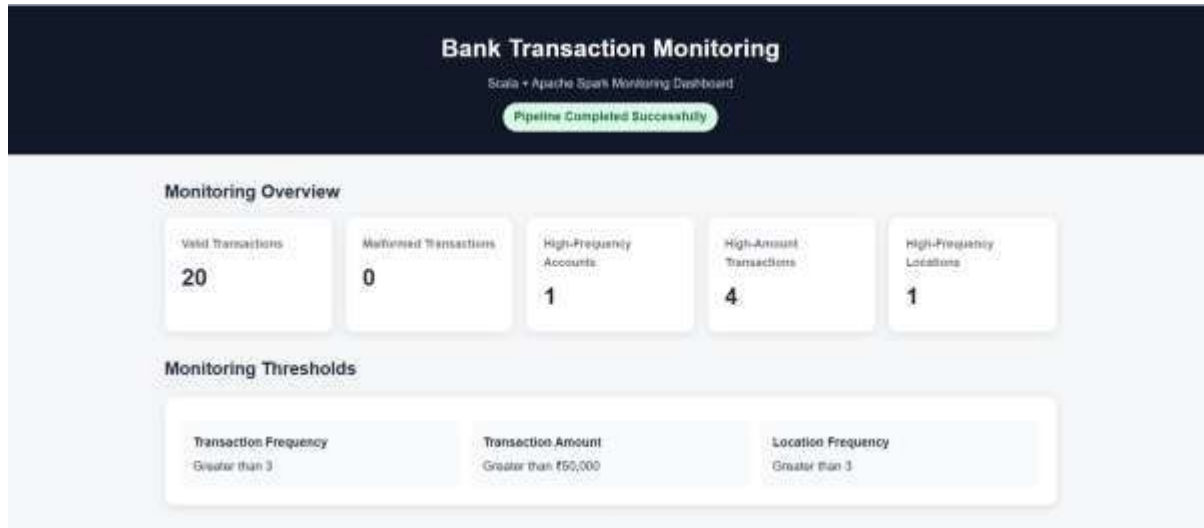


Figure 16: Bank Transaction Monitoring Dashboard showing final pipeline results

20. Final Results

20.1 Overall Results

The Bank Transaction Monitoring system successfully processed the sample dataset containing **20 valid transactions**.

The final monitoring results were:

Metric	Result
Valid Transactions	20
Malformed Transactions	0
High-Frequency Accounts	1
High-Amount Transactions	4
High-Frequency Locations	1

20.2 Detected Alerts

- **ACC001** — 6 transactions, exceeding the frequency threshold of 3.
- **TXN005** — ₹75,000
- **TXN009** — ₹60,000
- **TXN014** — ₹90,000
- **TXN015** — ₹1,20,000
- **Hyderabad** — 7 transactions, exceeding the location threshold of 3.

20.3 Processing Validation

The project successfully demonstrated:

- Batch transaction processing
- Stateful streaming
- Sliding-window processing
- Pair RDD aggregation
- DataFrame analytics
- Broadcast thresholds
- Accumulator-based validation
- Partition management
- spark-submit execution

21. Performance, Deployment and CI

21.1 Performance Considerations

The project uses Spark features that support efficient distributed processing:

- **Caching** reduces repeated computation of the transaction RDD.
- **Pair RDD aggregation** performs account and location-based counting.
- **Broadcast variables** efficiently distribute small monitoring thresholds.
- **Partition management** demonstrates control over parallel data processing.
- **DataFrames** provide structured aggregation and analytical processing.

21.2 Spark Submit Deployment

The final application was successfully executed using spark-submit in local mode.

The execution used:

```
spark-submit  
--class FinalMonitoringProcessor  
--master local[*]
```

This validates that the application can be packaged and executed as a Spark application rather than only being run through the development environment.

21.3 GitHub Actions CI

A GitHub Actions workflow was configured to automatically validate the Scala/Spark project.

The CI workflow performs:

Checkout Repository

↓

Set up Java 17

↓

Set up sbt

↓

Clean

↓

Compile

↓

Package

The workflow completed successfully, confirming that the project builds correctly in the CI environment.

21.4 Deployment Architecture

The project supports the following practical workflow:

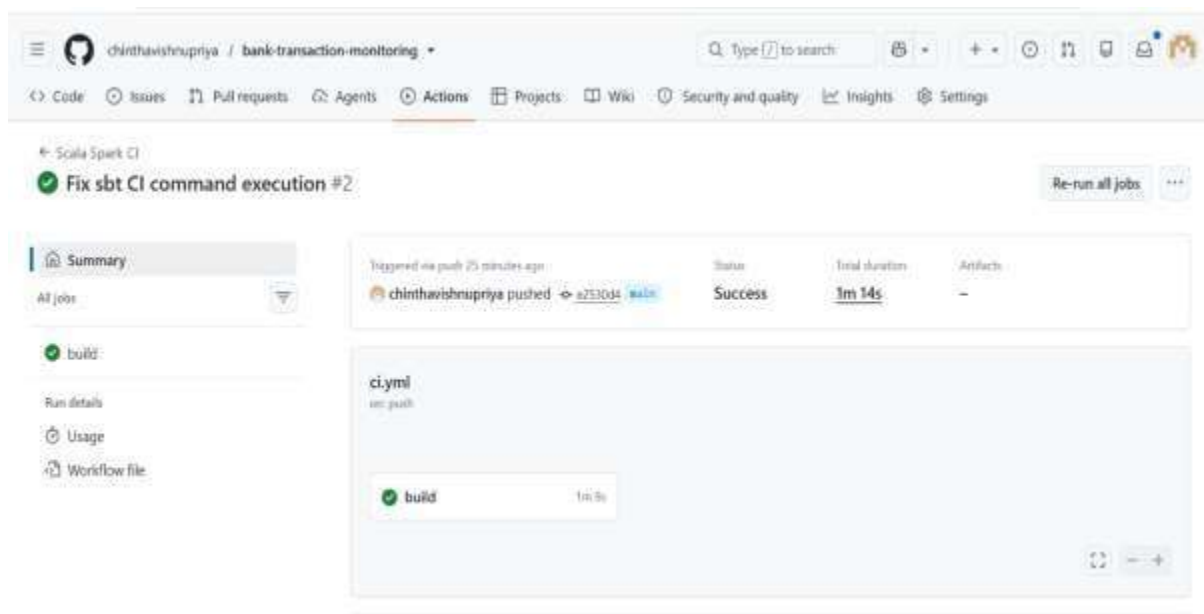
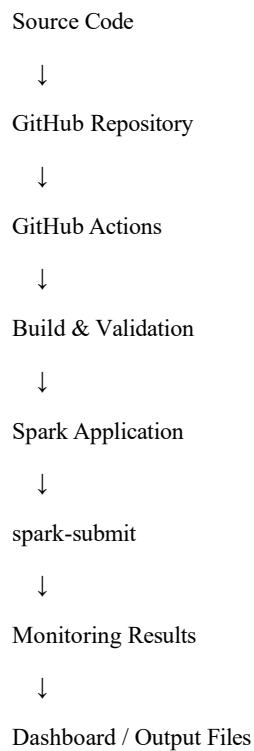


Figure 17. GitHub Actions CI workflow showing successful project validation

22.Conclusion

The Bank Transaction Monitoring project successfully demonstrates the application of Scala and Apache Spark to transaction-monitoring and distributed data-processing tasks.

The implemented system validates transaction records, performs account and location-based analysis, and identifies unusual activity using configurable monitoring thresholds.

The project demonstrates Pair RDD operations, narrow and wide transformations, stateful streaming with `updateStateByKey`, sliding-window processing, broadcast variables, accumulators, caching, partition management, DataFrame aggregations, RDD lineage and fault tolerance, spark-submit, GitHub Actions CI, and an HTML/CSS monitoring dashboard.

The final execution successfully processed 20 valid transactions and detected 1 high-frequency account, 4 high-amount transactions, 1 high-frequency location, and 0 malformed transactions.

The completed system provides a practical Spark-based monitoring workflow from input processing and distributed computation through validation, deployment, and result visualization.

- 1 high-frequency account
- 4 high-amount transactions
- 1 high-frequency location
- 0 malformed transactions